Future Headlines (WtG) – Codebase Walkthrough

Developer handoff reference

2026

Contents

1 Project overview

Future Headlines (working title *WtG*) is a real-time multiplayer web game about the near future of AI. Players write short **story directions** – a dated, near-future development (e.g. *"2031: an AI passes the bar exam in every US state"*). For each submission an LLM **"juror"**:

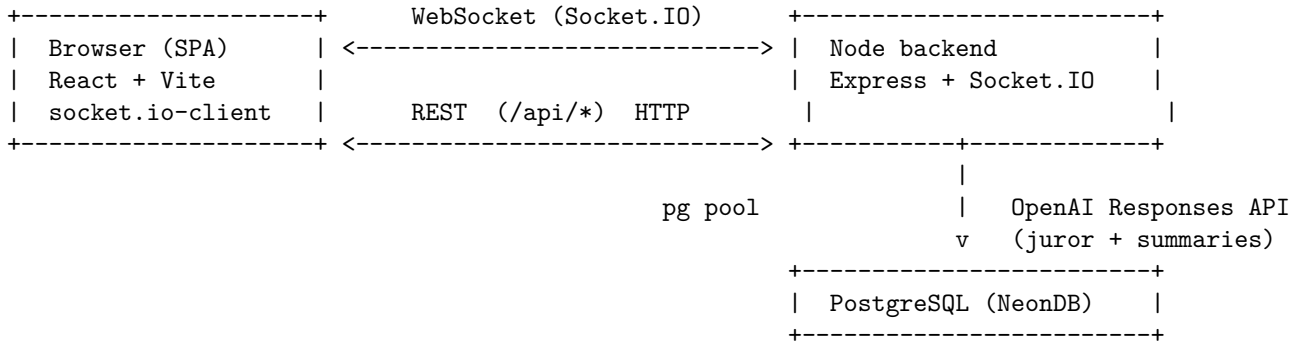
1. rates its **plausibility** (1-5),
2. classifies it into the three most relevant **"planets"** (thematic categories),
3. **links** it to the three most related earlier headlines, and
4. drafts **five newspaper-style variants**, one per plausibility band.

A **dice roll** then picks which of the five variants is actually published to the shared timeline, and the player is **scored** on plausibility, story connections, and which planet they targeted. A game runs as a sequence of timed rounds across a compressed ~20-year in-game timeline that begins seeded with 36 real 2022-2025 AI headlines.

Stack at a glance

- **Frontend:** React 18 + TypeScript + Vite + Tailwind CSS; real-time via `socket.io-client`. Static SPA.
- **Backend:** Node + Express + Socket.IO + TypeScript; PostgreSQL via `pg`; OpenAI **Responses API** for all LLM calls; `zod` for input validation.
- **Database:** PostgreSQL (hosted on NeonDB), plain-SQL migrations.

2 Architecture at a glance



- **REST** (`/api/*`) is used only for session lifecycle (create / join / rejoin / fetch). Everything during a live game flows over **Socket.IO**.
- The backend owns all game state and timing. A singleton **game loop** drives phase transitions and broadcasts state to each session's room. Clients are thin: they render whatever the server sends and emit player actions.
- The **juror** and **summary** generators are the only LLM touch-points; both go through one Responses-API client wrapper.

3 Repository structure

Path	Purpose
`backend/`	Express + Socket.IO server (the game engine, scoring, LLM, DB).
`frontend/`	React + Vite single-page app (the player UI).
`experiments/`	Research artifacts (inter-rater analysis, prompt experiments). Not part of the running app.
`Report/`	The dissertation (LaTeX). Not part of the running app.
`tsconfig.base.json`	Shared TypeScript base config.
`.gitignore`	Ignores `node_modules/`, `dist/`, `.env*`, logs, coverage.

There is no root README; this document is the entry point.

4 Running it locally

Prerequisites: Node 20+, a PostgreSQL database (local or a NeonDB URL), and an OpenAI API key.

1. Backend env – create `backend/.env` (placeholders shown; never commit real secrets):

```
DATABASE_URL=postgres://USER:PASS@HOST:5432/DBNAME
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-5.2              # optional, this is the default
OPENAI_BASE_URL=https://api.openai.com   # optional
PORT=3001                         # optional
FRONTEND_URL=http://localhost:5173       # optional, used for CORS
GAME_TEST_MODE=false              # set true to compress all timings by 1/16
```

2. Frontend env – optional `frontend/.env` (defaults to localhost):

```
VITE_BACKEND_URL=http://localhost:3001
```

3. Install, migrate, run (two terminals):

```
cd backend  && npm install && npm run migrate && npm run dev   # serves :3001
cd frontend && npm install && npm run dev                      # serves :5173
```

Open http://localhost:5173. The Vite dev server proxies `/api` and `/socket.io` to `:3001`.

Fast playtesting: start the backend with `GAME_TEST_MODE=true` to scale every real-time duration by 1/16 (a full game runs in ~3 minutes; the submission cooldown drops from 90s to ~6s). The in-game timeline still spans the full ~20 years. You will see `GAME_TEST_MODE active -- all durations scaled by 1/16` in the logs.

Useful scripts (per package): `npm run dev` (watch), `npm run build` (`tsc`, or `tsc && vite build` for the frontend), `npm test` (backend Jest), `npm run migrate` (apply DB migrations), `npm run lint`, `npm run format`.

5 Game flow and rules

5.1 Phase machine

The game progresses through phases driven entirely by the backend game loop (`backend/src/game/gameLoop.ts`):

```
WAITING --> TUTORIAL --> PLAYING <--> BREAK --> ... --> FINISHED
```

- **WAITING** – lobby; host waits for >=2 players, then starts.
- **TUTORIAL** – ~3 min; the in-game clock is frozen (speed ratio 0). The 36 seed headlines **drip-feed** into the timeline (one every few seconds) so players start with context. No submissions allowed.
- **PLAYING** – ~8 min per round; players submit story directions. The in-game clock advances fast (see below).
- **BREAK** – between rounds (~3/5/3 min); clock frozen. After round 2 a **round recap** summary is generated.
- **FINISHED** – no timer; the end-of-game **narrative** (fictional first-person reports) is generated.

A game is **4 rounds** by default (`max_rounds`), each round being one PLAYING phase followed by a BREAK (except the last). `GAME_TEST_MODE` scales all of these durations by 1/16.

5.2 In-game timeline

The published timeline spans ~20 years. Each PLAYING round advances the in-game clock at a high `timelineSpeedRatio` (computed per round so the rounds together cover the full span; earlier rounds move slower). The clock is frozen during TUTORIAL/BREAK/FINISHED. `computeInGameNow()` in `backend/src/game/inGameTime.ts` derives the current in-game date from the phase start, real elapsed time, and the ratio – **clamped to the phase window** so a long-stale phase can never overflow the JS `Date` range.

5.3 Headline lifecycle (what happens on a submission)

```
player submits story direction
      |
      v
[validate] phase == PLAYING, 90s per-player cooldown, zod schema
      |
      v
[context] fetch the most recent N headlines (N = 36 = seed count)   <-- rolling window
      |
      v
[juror]  OpenAI Responses API -> plausibility(1-5), top-3 planets,
         3 linked headlines (STRONG/WEAK), 5 headline variants
      |
      v
[dice]   roll 1-100 -> band 1-5 -> pick that variant as the published headline
      |
      v
[store]  INSERT into game_session_headlines (all 5 bands, dice, planets, links, ...)
      |
      v
[score]  async: baseline + plausibility + connection + planet band; update totals;
         bump global planet usage
      |
      v
[broadcast]  headline:new (to all)  +  leaderboard:update (scores + planet panels)
```

The submission callback returns right after the headline is stored; **scoring is asynchronous** and arrives via a separate `leaderboard:update` broadcast.

5.4 Scoring

A headline's score is the sum of four parts (defaults in `backend/src/game/scoringTypes.ts` `DEFAULT_SCORING_CONFIG`):

Component	Rule	Points
Baseline	every submission	+1
Plausibility	uses the juror's level: 3 = "sweet spot" (A1); 2 or 4 = near (A2); 1 or 5 = off	+2 / +1 / 0
Connection	count of **distinct other players** among your STRONG-linked headlines (0-3), scale `[0,1,4,9]`	0 / 1 / 4 / 9
Planet	the band your primary planet sits in (by global usage rank)	+2 / +1 / +0

Notes:

- Plausibility scoring uses the **juror's assessed level**, not the dice roll. The dice roll only decides which of the five drafted variants is *displayed*.
- Connection scoring resolves the authors of STRONG-linked headlines by DB lookup, so it is unaffected by the juror context window.

5.5 Planet usage system

There are **9 planets** (`backend/src/game/planets.ts`): MERCURY, VENUS, EARTH, MARS, JUPITER, SATURN, URANUS, NEPTUNE, PLUTO – each a thematic category with a description (and frontend keywords/colours in `frontend/src/lib/planets.ts`).

- Usage is **global per session** (`game_sessions.planet_usage_global`). Each scored headline increments its **primary (rank-1) planet** by 1.
- Planets are ranked least-used -> most-used and split into three **bands**: least-used 3 give **+2**, middle 3 give **+1**, most-used 3 give **+0**. Band membership is the same for every player (shared global usage, with a fixed canonical tie-break).
- Each player also has a **stable random permutation** (`session_players.planet_usage_state`, the "ordinals") used only to **shuffle the order within each band** per player – so the panel doesn't look identical to everyone, while the bands themselves stay shared.

Logic lives in `backend/src/game/planetUsage.ts` (`computeBandMembership`, `computePlanetPanel`, `applyGlobalPlanetScori`
The older per-player "priority planet" module `planetWeighting.ts` is **deprecated** and no longer wired in.

6 Backend reference

All paths are under `backend/src/`.

6.1 Entry and wiring

- **`server.ts`** – builds the Express app + HTTP server + Socket.IO server. CORS origin = `FRONTEND_URL` (default `http://localhost:5173`); listens on `PORT` (default 3001). Mounts `GET /health`, the sessions router at `/api`, the juror router at `/api/juror`; calls `gameLoopManager.setSocketIO(io)` and `setupLobbyHandlers(io)`; handles graceful shutdown (stops all game loops).

6.2 REST routes

- **`routes/sessions.ts`** (mounted at `/api`):
  - `POST /api/sessions` – create a session (defaults: 8 play min, 3 break min, 4 rounds) + host player; returns join code + player id.
  - `POST /api/sessions/:joinCode/join` – join as a new player (validates code + unique nickname).
  - `POST /api/sessions/:joinCode/rejoin` – recover an existing player by nickname, **regardless of phase** (cross-device / refresh recovery).
  - `GET /api/sessions/:joinCode` – fetch session + players.
- **`routes/juror.ts`** (mounted at `/api/juror`): `POST /api/juror/evaluate` (run the juror on a story direction – used for experiments/manual testing) and `GET /api/juror/health`.

6.3 Socket layer – `socket/lobbyHandlers.ts`

The heart of live gameplay. Handles inbound events and emits outbound broadcasts (see the **Real-time event contract** section). It also contains:

- the per-player **90s submission cooldown** (in-memory `Map`; ~6s in test mode),
- `JUROR_HISTORY_WINDOW` – the rolling context window size (= `SEED_HEADLINES.length` = 36),
- `getSessionState()` – builds the full `SessionState` (players, scores, planet panels, in-game clock) returned to clients and broadcast on phase changes,
- `deriveUniqueOtherAuthorCount()` – connection-scoring helper (counts distinct other authors among STRONG links).

6.4 Game loop – `game/gameLoop.ts` (+ `inGameTime.ts`)

A singleton `gameLoopManager` keyed by session. Owns phase transitions, per-round timers, the seed drip, the in-game clock, and `broadcastGameState()`. `TIME_SCALE` (1 normally, 1/16 in `GAME_TEST_MODE`) scales all durations. Round speed ratios come from `computeRoundSpeedRatio()`.

6.5 Scoring modules

- **scoring.ts** – pure, DB-free score functions (baseline / plausibility / connection / total).
- **scoringService.ts** – applyHeadlineEvaluation() runs the scoring DB transaction (locks the session row, reads/writes global planet usage, persists the breakdown, returns the leaderboard with per-player planet panels).
- **scoringTypes.ts** – types + DEFAULT_SCORING_CONFIG (tune scoring here).
- **planetUsage.ts** – band-based global planet system (above). **planetWeighting.ts is deprecated.**

6.6 LLM integration

- **llm/openaiResponsesClient.ts** – thin wrapper over the OpenAI Responses API; enforces a JSON schema for structured output; model from OPENAI_MODEL (default gpt-5.2).
- **game/jurorService.ts + llm/jurorPrompt.ts** – the juror. Output schema: PLAUSIBILITY {band, label, rationale}, PLANETS (top-3 with rank + rationale), LINKED (exactly 3, STRONG/WEAK + rationale), HEADLINES (band1..band5 variants). Validated strictly (exactly 3 linked / 3 planets).
- **game/headlineTransformationService.ts** – orchestrates juror -> dice -> selected variant.
- **game/diceRoll.ts** – BAND_BOUNDARIES map a 1-100 roll to a band with distribution **10 / 35 / 40 / 12 / 3** (%), i.e. band 3 "plausible" is most likely.
- **game/summaryService.ts + llm/summaryPrompt.ts** (round recap) and **llm/narrativePrompt.ts** (end-of-game first-person reports). Both **exclude Archive/seed headlines**.

6.7 Planets and seeds

- **game/planets.ts** – the 9 planets + descriptions (passed to the juror).
- **game/seedHeadlines.ts** – the 36 real 2022-2025 headlines drip-fed during TUTORIAL by the Archive system player. (This count also sets the juror window size.)

6.8 Database access

- **db/pool.ts** – pg pool (max 20, short idle timeout, keep-alive) tuned for NeonDB's idle disconnects.
- **db/migrate.ts** – runs db/migrations/*.sql in filename order inside transactions, recording applied files in a schema_migrations table (each runs once). Invoke with npm run migrate.

7 Frontend reference

All paths are under frontend/src/.

7.1 Entry, routing, stack

- **main.tsx** – React 18 entry, wraps App in BrowserRouter.
- **App.tsx** – routes: / (create session), /join/:joinCode (pages/JoinByLinkPage.tsx, accept an invite link), /lobby/:joinCode (host or join lobby). Persists {joinCode, playerId, isHost} in localStorage, makes the REST calls (with a rejoin fallback when a started game blocks a normal join), wires up useSocket, and renders by phase.
- **vite.config.ts** – dev proxy of /api + /socket.io to :3001. **vercel.json** – SPA rewrite.
- Env: VITE_BACKEND_URL (default http://localhost:3001).

7.2 Real-time state – hooks/useSocket.ts

The single hub for all Socket.IO traffic. Holds sessionState, headlines, roundSummary, finalSummary, exposes actions (joinLobby, startGame, submitHeadline, loadHeadlines, requestSummary, …), and defines the shared client types: Player (incl. planetPanel: PlanetPanelEntry[]), Headline (incl. selectedBand for typography), SessionState, ScoreBreakdown, RoundSummary/FinalSummary outputs.

7.3 Time hooks

- `useInGameNow.ts` – locally ticks the in-game date between server snapshots (only while PLAYING).
- `usePhaseTimer.ts` – `MM:SS` countdown to phase end (header).
- `useGameTimeProgress.ts` – cumulative game progress used to scale the score bars.

7.4 Layout and screens

- `components/GameLayout.tsx` – master responsive layout. Desktop is 3 columns (left: score chart + in-game date + scoring legend; centre: headline feed + input; right: **planet usage panel**, which swaps to the **round summary** during a BREAK). Mobile stacks these. Renders lobby / game / end by phase.
- `HostLobby.tsx` / `JoinLobby.tsx` – pre-game screens (invite link, start button, player list).
- `GameEnd.tsx` – final leaderboard, full headline feed, and the AI "experience reports", plus the **PDF download** (see below).

7.5 Gameplay components

- `HeadlineFeed.tsx` – the timeline. Font size/weight scale with the headline's plausibility band (`BAND_TEXT`: small/light for "inevitable" up to large/bold for "preposterous"); a coloured left border + a planet chip indicate the primary planet; Archive entries are styled as history; hover shows the score breakdown.
- `HeadlineInput.tsx` – 280-char submit form with the cooldown countdown.
- `PlanetUsagePanel.tsx` – the 9 planets grouped into the three bands (+2/+1/+0), each row showing name, keywords, and usage count.
- `ScoreCard.tsx` (rules legend), `ScoreBarChart.tsx` (stacked leaderboard bars), `PersonalScore.tsx`, `PlayerList.tsx`, `InGameDate.tsx`, `RoundSummary.tsx`, `GameStatus.tsx` (phase badge + round + countdown), and `ui.tsx` primitives (`Card`, `Button`, `Badge`, `SectionTitle`).
- `lib/planets.ts` – planet keyword tags + Tailwind colour classes (written out in full so Tailwind's scanner keeps them).

7.6 Game-end PDF export

`GameEnd.tsx` builds a downloadable PDF with **jsPDF**: the leaderboard chart is captured as an image (html2canvas) and the headline timeline + experience reports are written as **native selectable text** with page-break handling. Filename: `future-headlines-<joinCode>-<YYYY-MM-DD>.pdf`. (This is the in-app game recap, separate from this handoff document.)

8 Real-time event contract

This is the integration seam between frontend and backend (`socket/lobbyHandlers.ts` <-> `hooks/useSocket.ts`).

Client -> server (request/ack):

Event	Payload	Returns
`lobby:join`	`{joinCode, playerId}`	`{success, state}`
`lobby:get_state`	`{joinCode}`	`{success, state}`
`lobby:start_game`	`{joinCode}`	`{success, state}`
`lobby:leave`	–	–
`headline:submit`	`{joinCode, headline}`	`{success, headline, cooldownMs}`
`headline:get_feed`	`{joinCode, roundNo?}`	`{success, headlines}`
`round:get_summary`	`{joinCode, roundNo}`	`{success, status, summaryType, summary}`

Server -> client (broadcast to the session room):

Event	Payload	When
`game:state`	full `SessionState`	phase transitions / state refresh
`headline:new`	a `Headline`	each accepted submission
`leaderboard:update`	`{leaderboard[], lastScoredHeadline}`	after scoring (carries per-player planet panels)
`round:summary`	`{roundNo, status, summary}`	round recap ready
`game:final_summary`	`{status, summary}`	end-of-game narrative ready
`lobby:player_joined`	`{playerId, player}`	someone joins the lobby
`lobby:game_started`	`{state}`	host starts the game

9 Data model

Key tables (see `backend/db/migrations/`):

- **`game_sessions`** – one row per game: `join_code`, `status/phase`, timing columns, in-game clock (`in_game_start_at`, `timeline_speed_ratio`), and **`planet_usage_global`** (JSONB, the shared usage counts).
- **`session_players`** – roster: `nickname`, `is_host`, **`is_system`** (the Archive player), `total_score`, and **`planet_usage_state`** (JSONB, now the per-player ordinal permutation).
- **`game_session_headlines`** – one row per submission (and per seed): the story direction, all five band variants, `dice_roll` / `selected_band` / `selected_headline`, juror plausibility + rationale, the three planets, `linked_headlines` (JSONB), the full scoring breakdown, `in_game_submitted_at`, and LLM request/response logs.
- **`round_summaries`** – generated recaps / narrative (status + JSONB payload + `summary_type`).
- **`schema_migrations`** – which migrations have run.

Migrations 001-014 (one line each):

File	Purpose
001_init	sessions + players, status enum, host FK
002_game_timing	phase/timing columns; state-transition audit table
003_headlines	`game_session_headlines` base table
004_scoring	player `total_score` + `planet_usage_state`; headline score columns
005_headline_transformation	dice/band/variant + juror columns (links, planets, rationale)
006_llm_request_response	store LLM request/response JSON
007_scoring_columns	re-add scoring columns idempotently
008_round_summaries	`round_summaries` table
009_headline_ingame_time	`in_game_submitted_at`
010_system_player	`is_system` (Archive)
011_tutorial_phase	TUTORIAL added to the phase/status set
012_round_format	defaults: 8 play minutes, 4 rounds
013_summary_type	`summary_type` (recap vs narrative)
014_planet_usage_global	`game_sessions.planet_usage_global` for band-based scoring

10 Testing

Backend tests use **Jest** (`backend/tests/`), ~249 tests:

- `tests/game/` – `scoring`, `scoringService`, `planetUsage`, `planetWeighting`, `diceRoll`, `gameLoop`, `gameLoopManager`, `jurorService`.
- `tests/socket/` – `lobbyHandlers`, `headlineHandlers` (the submit flow; mocks `pool.query`).
- `tests/routes/` – `jurorRoutes`. `tests/llm/` – `openaiResponsesClient`.

Run with `npm test` (or `npm run test:watch`) from `backend/`. The frontend currently has **no** test suite.

11 Gotchas and non-obvious behaviours

- `GAME_TEST_MODE=true` scales every real-time duration by 1/16 (and cooldown to ~6s). The in-game timeline still spans the full ~20 years. Essential for testing a full game quickly; has no effect unless set.
- **Scoring is asynchronous.** `headline:submit`'s ack returns after the headline is stored; the score and re-ranked planet panels arrive later via `leaderboard:update`. Don't assume scoring is done at ack time.
- **Planet bands are global; ordinals are per-player.** Every player sees the same planets in each band (driven by shared global usage); only the order *within* a band is shuffled per player.
- **The juror only sees the last 36 headlines** (`JUROR_HISTORY_WINDOW`), a rolling window – so plausibility and connection-linking stay focused on recent context as the timeline grows. Summaries, by contrast, use the full history (minus Archive).
- **Dice roll != plausibility score.** The roll only chooses which drafted variant is published; scoring uses the juror's plausibility level.
- **In-game clock is clamped** to the phase window (`inGameTime.ts`) so a stale/overrun phase can't overflow JS `Date` (this previously caused a `RangeError`).
- **Archive is a system player** (`is_system = true`): it injects the seed headlines and is filtered out of leaderboards and summaries. Don't surface it as a real player.
- **The submission cooldown is in-memory** – it resets if the backend restarts.
- **Summaries generate asynchronously** with a status (`generating` -> `completed`/`error`); they don't block phase transitions.
- **`planetWeighting.ts` is deprecated** – the live system is `planetUsage.ts`.

12 "Where to change X" cookbook

You want to change…	Edit
Scoring weights (baseline / plausibility / connection / planet)	`backend/src/game/scoringTypes.` -> `DEFAULT_SCORING_CONFIG`
Dice band probabilities (currently 10/35/40/12/3)	`backend/src/game/diceRoll.ts` -> `BAND_BOUNDARIES`
Round count, play/break minutes, round speed ramp	`backend/src/game/gameLoop.ts` (`BREAK_SCHEDULE`, durations, `ROUND_SPEED_WEIGHTS`)
How many past headlines the juror sees	`backend/src/socket/lobbyHandle` -> `JUROR_HISTORY_WINDOW`
Juror / summary / narrative prompts	`backend/src/llm/jurorPrompt.ts` `summaryPrompt.ts`, `narrativePrompt.ts`
LLM model	`OPENAI_MODEL` env var (default `gpt-5.2`)
Planets, descriptions, keywords, colours	`backend/src/game/planets.ts` + `frontend/src/lib/planets.ts`
The seed (Archive) headlines	`backend/src/game/seedHeadlines`
Headline typography by band	`frontend/src/components/Headli` -> `BAND_TEXT`
Add a DB column / table	new `backend/db/migrations/0NN_*.sq` then `npm run migrate`

13 Glossary

- **Story direction** – the dated near-future development a player submits.
- **Juror** – the LLM that rates plausibility, classifies planets, links headlines, and drafts variants.
- **Plausibility level** – the juror's 1-5 rating (1 inevitable … 5 preposterous); 3 is the scoring sweet spot.
- **Dice band** – a 1-5 value from the dice roll that selects which drafted variant is published.
- **Planet** – one of 9 thematic categories a headline can belong to.
- **Planet band** – the +2 / +1 / +0 scoring tier a planet sits in, by global usage rank.
- **Archive** – the system player that seeds the timeline with 36 real headlines.
- **Rolling window** – the most recent 36 headlines shown to the juror.
- **In-game clock** – the compressed ~20-year timeline date, advanced during PLAYING.